

AIエージェント「Maverick-Agent」開発指示書

1. プロジェクト概要

本プロジェクトは、オープンソースのAIエージェントフレームワーク「Maverick」をベースとし、Devinのような高度なGUIを備えたWebアプリケーション「Maverick-Agent」を開発することを目的とする。

ユーザーはWebブラウザを通じて、タスクの指示、進捗の確認、コードの編集、エージェントとの対話を行うことができる。開発環境はDocker Composeで一括起動可能とし、WindowsおよびmacOSでのクロスプラットフォーム動作を保証する。

2. システムアーキテクチャ

Maverick-Agentは、以下の4つのサービス(Dockerコンテナ)で構成されるマイクロサービスアーキテクチャを採用する。これにより、各機能の独立性が保たれ、開発とメンテナンスが容易になる。

サービス名	役割	主要技術・ツール
frontend	ユーザーインターフェース(GUI)を提供。全ての操作の窓口となるWebアプリケーション。	React (Next.js or Vite), TypeScript, Tailwind CSS, Zustand
backend	エージェントのコアロジックを担うAPIサーバー。Maverickとの連携、LLMの切り替え、Git操作などを処理する。	Python, FastAPI, Maverick, GitPython, WebSocket
code-editor	ブラウザ上で動作するVSCode環境を提供。ファイルの編集、ターミナルの操作を行う。	codercom/code-server
debug-server	デバッグ用のMinecraft (MCP) サーバー。エージェントが特定のタスクをデバッグする際のターゲットとして利用。	itzg/minecraft-server

構成図

```
graph TD
    subgraph "ユーザーのブラウザ"
        A[User]
    end
```

```
subgraph "Docker環境 (ホストマシン)"
  B[Frontend (React)] -- HTTP/WebSocket --> C[Backend (FastAPI)]
  B -- iframe --> D[Code Editor (code-server)]
  C -- ファイル操作 --> V[Shared Volume: ./workspace]
  D -- ファイル操作 --> V
  C -- デバッグ命令 --> E[Debug Server (Minecraft)]
end
```

```
subgraph "外部サービス"
  F[Gitリポジトリ (GitHub, etc.)]
  G[LLM APIs (Qwen, DeepSeek, etc.)]
end
```

```
A -- "操作" --> B
C -- "Gitコマンド" --> F
C -- "APIコール" --> G
```

```
style B fill:#d4f1f4,stroke:#05445e
style C fill:#9ad3bc,stroke:#18978f
style D fill:#f8d7da,stroke:#721c24
style E fill:#e9c46a,stroke:#2a9d8f
style V fill:#e0e0e0,stroke:#666,stroke-dasharray: 5 5
```

3. 機能要件詳細

3.1. フロントエンド (GUI) - frontendサービス

画面構成: 3ペイン構成のダッシュボードを基本とする。

- 左ペイン: ナビゲーション & 設定
 - ファイルエクスプローラー: code-editorと連携し、ワークスペースのファイルツリーを表示。クリックでファイルを開く。
 - 設定ボタン: クリックすると設定モーダルが開く。
- 中央ペイン: メインワークスペース
 - **VSCode**エディタ: code-editorサービスをiframeで埋め込み表示。コード編集、ターミナル操作をここで行う。
- 右ペイン: エージェントコントロールパネル
 - タスクプラン表示エリア:
 - Devinのように、大元のタスクがサブタスクに分解されてリスト表示される。

- 各サブタスクのステータス(待機中, 実行中, 完了, エラー)がアイコンで視覚的に分かるようにする。
- チャット/ログ表示エリア:
 - エージェントの思考プロセス("Thinking...")、実行コマンド、結果、エラーログがリアルタイムで表示される。
- プロンプト入力欄:
 - ユーザーがエージェントへの指示を入力する。
 - モード切替トグル: 「**Agent**(自律実行)」モードと「**Ask**(要確認)」モードを切り替える。
 - **Agent**モード: エージェントは確認なしでタスクを続行する。
 - **Ask**モード: 不明点や重要な操作の前に、エージェントがユーザーに許可を求める。

個別機能:

- 設定モーダル:
 - **Git連携タブ:**
 - リポジトリURL入力欄。
 - 認証方法選択 (SSH / HTTPS)。
 - SSH秘密鍵またはHTTPSアクセストークンを入力・保存するフォーム。保存された情報はbackendに安全に送信される。
 - モデル設定タブ:
 - ドロップダウンメニューで以下のLLMを選択可能にする。
 - Qwen3
 - DeepSeek
 - Qwen2.5
 - Maverick (Default)
 - 選択したモデルはbackendに通知され、以降のエージェントの思考に利用される。

3.2. バックエンド - backendサービス

APIエンドポイント & WebSocket:

種別	Path	説明
POST	/api/task/start	フロントエンドからタスク指示を受け取り、エージェントの実行を開始する。
GET	/api/settings	現在の設定(モデル、Git情報)を取得する。

POST	/api/settings	設定を更新する。
POST	/api/git/clone	指定されたリポジトリをクローンする。
WebSocket	/ws/agent	エージェントの状態(思考、タスク進捗、ログ)をリアルタイムでフロントエンドに送信する。
WebSocket	/ws/ask	エージェントがユーザーに質問を送信し、ユーザーからの返信を受信する。

コアロジック:

- **タスク管理 (Maverick連携):**
 - ユーザーからの指示をMaverickのタスク分解機能に入力し、サブタスクのリストを生成する。
 - 生成されたサブタスクを順番に実行し、各ステップの進捗、コマンド実行結果、思考プロセスをWebSocket経由で逐次送信する。
- **LLMクライアント:**
 - 設定に応じて、指定されたLLMを呼び出すクライアントを動的に切り替えるロジックを実装する。
- **ファイルシステム連携:**
 - ホストマシンと共有されたボリューム (./workspace) を介して、code-editorとファイルの読み書きを同期させる。エージェントが生成・編集したコードが即座にVSCodeエディタに反映されるようにする。

3.3. VSCode Editor - code-editorサービス

- codercom/code-serverのDockerイメージをベースとする。
- 起動時に、共有ボリューム (/home/coder/project) をデフォルトのワークスペースとして開くように設定する。
- 認証は無効化するか、docker-compose.yml内で固定のパスワードを設定して利便性を高める。
- frontendの中央ペインにiframeとして埋め込まれる。

3.4. デバッグサーバー - debug-serverサービス

- Minecraftサーバーのデバッグを想定し、itzg/minecraft-serverイメージを利用する。
- EULA=TRUEを環境変数で設定し、自動で起動するようにする。
- サーバーのポート (25565) はbackendコンテナからアクセスできるようにする。
- サーバーのコンソールログは、必要に応じてbackend経由でフロントエンドにストリーミング

グできるようにする。

4. インフラ・デプロイメント (Docker)

プロジェクトのルートに以下のdocker-compose.ymlファイルを配置する。このファイル一つで、すべての開発環境が整う。

```
# docker-compose.yml
version: '3.8'

services:
  # 1. フロントエンド (GUI)
  frontend:
    build:
      context: ./frontend
      dockerfile: Dockerfile
    ports:
      - "3000:3000" # ブラウザからアクセス
    volumes:
      - ./frontend:/app # 開発時のホットリロード用
      - /app/node_modules # node_modulesをコンテナ内に保持
    restart: unless-stopped
    depends_on:
      - backend

  # 2. バックエンド (エージェントコア)
  backend:
    build:
      context: ./backend
      dockerfile: Dockerfile
    volumes:
      - ./backend:/app
      - ./workspace:/app/workspace # ワークスペースを共有
    ports:
      - "8000:8000" # APIサーバー
    environment:
      # .envファイルからAPIキーを読み込む
      - OPENAI_API_KEY=${OPENAI_API_KEY}
      - QWEN_API_KEY=${QWEN_API_KEY}
      - DEEPSEEK_API_KEY=${DEEPSEEK_API_KEY}
```

```
env_file:
  - .env
restart: unless-stopped
```

3. VSCodeエディタ

```
code-editor:
  image: codercom/code-server:latest
  ports:
    - "8080:8080"
  volumes:
    - ./workspace:/home/coder/project # ワークスペースをマウント
  environment:
    - PASSWORD=maverick # 固定パスワードを設定
    - SUDO_PASSWORD=maverick
  restart: unless-stopped
```

4. デバッグ用Minecraftサーバー

```
debug-server:
  image: itzg/minecraft-server
  ports:
    - "25565:25565"
  environment:
    EULA: "TRUE"
    TYPE: "PAPER" # 軽量なサーバータイプを選択
  volumes:
    - minecraft-data:/data
  restart: unless-stopped
```

```
volumes:
  minecraft-data:
```

起動手順:

1. プロジェクトのルートディレクトリに.envファイルを作成し、各種APIキーを記述する。
.env file
OPENAI_API_KEY=sk-...
QWEN_API_KEY=your-qwen-key
DEEPSEEK_API_KEY=your-deepseek-key

2. 各サービス(frontend, backend)のディレクトリにDockerfileを準備する。
3. ターミナルで以下のコマンドを実行するだけですべてのサービスが起動する。
`docker-compose up --build`
4. ブラウザで `http://localhost:3000` にアクセスすると、Maverick-AgentのGUIが表示される。

5. 推奨開発ステップ

1. **Step 1: 環境構築と疎通確認**
 - `docker-compose.yml`を完成させ、4つのサービスが正常に起動することを確認する。
 - backendとcode-editor間で./workspaceボリュームが正しく共有され、一方でのファイル変更がもう一方に反映されることを確認する。
2. **Step 2: バックエンドAPIとエージェントの基本動作**
 - FastAPIで基本的なAPIエンドポイントとWebSocketを実装する。
 - Maverickフレームワークを組み込み、単純なタスク(例: "workspaceにhello.txtを作成する")をAPI経由で実行できるようにする。
3. **Step 3: フロントエンドUIの構築**
 - Reactで3ペインのレイアウトを作成する。
 - code-editorサービスをiframeで埋め込む。
 - チャット欄やタスク表示エリアのUIコンポーネントを静的に作成する。
4. **Step 4: フロントエンドとバックエンドの連携**
 - タスク実行とリアルタイム表示: フロントエンドからタスクを送信し、WebSocket経由で進捗(思考ログ、タスクステータス)を受け取り、UIにリアルタイムで表示する。
 - 設定機能: 設定モーダルから入力された情報をbackendに送信し、保存・反映させる。
 - **Askモード**: エージェントからの質問をWebSocketで受け取り、ユーザーが回答を入力できるUIを実装する。
5. **Step 5: 統合テスト**
 - 具体的なシナリオ(例: "GitHubからリポジトリをクローンし、特定のファイルのバグを修正して、テストを実行する")を通して、システム全体の動作をテストする。
 - WindowsとmacOSの両環境で`docker-compose up`が問題なく動作することを確認する。